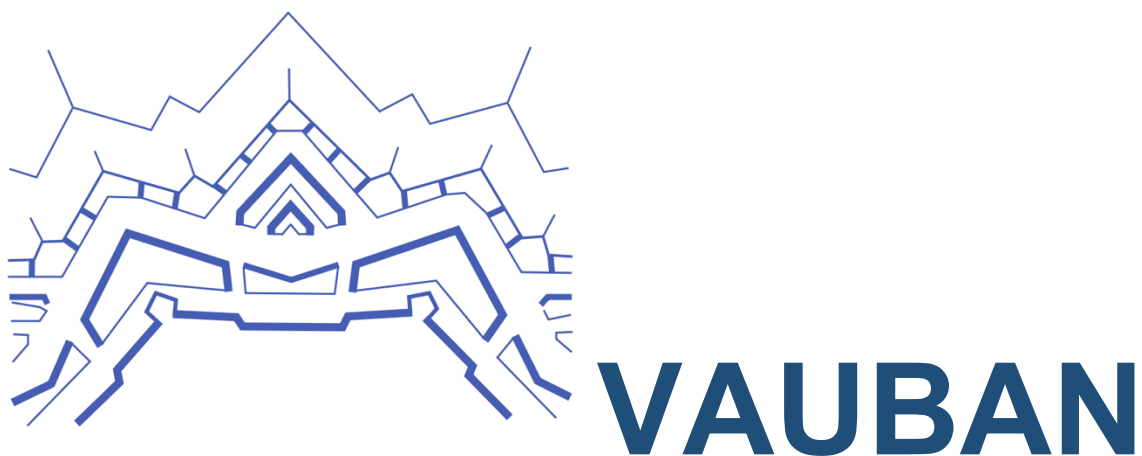




Privileged Access Management — Open Source Bastion

FUNCTIONAL ANALYSIS

Complete BA Documentation — Version 1.0

Author: Richard Ben Aleya

Status: Documentation — Reference Implementation

Document Control

Version	Description	Date	Editor
1.0	Initial Version	12/04/2026	Richard Ben Aleya

This document is the complete functional analysis of the Vauban software. It covers all functional domains of the system: authentication and identity management, asset management, access control, sessions and recording, and infrastructure.

Section	Functional Domain
1	System Overview
2	Actors and Personas
3	Authentication and Account Management
4	Asset Management (SSH & RDP)
5	Access Control and Groups
6	Sessions and Recording
7	Infrastructure and Administration
8	Cross-Cutting Non-Functional Requirements
9	Consolidated API Reference
10	Business Glossary

1. System Overview

1.1 Description

Vauban is an open-source Privileged Access Management (PAM) bastion designed to protect and control access to critical computing assets in enterprise, industrial, and defense environments. It draws on the privilege-separation architecture of OpenSSH and leverages the Capsicum sandboxing mechanism of FreeBSD to enforce strong isolation between its internal components.

The system is built on a multi-process architecture in which each service runs under a dedicated system account and can only access the resources strictly required for its role. Inter-service communication is handled exclusively through IPC channels pre-established by the orchestrator, with no inter-service network exposure.

1.2 Functional Scope

- User authentication via username/password with optional TOTP-based multi-factor authentication (MFA)
- Two-layer access control: role-based access control (RBAC) and instance-level access rules by asset group
- Interactive SSH access via a web-based terminal
- Interactive RDP access via a web-based remote desktop with H.264 video encoding
- Full session recording for SSH and RDP
- Automatic password redaction in SSH session recordings
- Just-In-Time (JIT) approval workflow for high-risk access requests
- Automated TLS certificate management via ACME without requiring port 80
- Command-line administration tools (CLI) for bootstrapping and maintenance

1.3 Service Architecture

Service	Responsibilities
vauban-supervisor	Process orchestration, watchdog, TCP connection brokering, and recording file delegation. The only non-sandboxed component.
vauban-web	HTTPS server, REST API, SSH/RDP WebSocket handling, web interface. Single entry point for end users.
vauban-auth	Secure password hashing and verification. Stateless, no database.
vauban-access	RBAC (role policy engine) and instance-level access rules. The sole authorization decision-maker in production.
vauban-vault	Secret encryption and decryption (SSH/RDP credentials, TOTP secrets). Encryption keys held in memory only.
vauban-audit	Persistent session recording for SSH and RDP. File access delegated by the supervisor via file descriptor passing.

Service	Responsibilities
vauban-proxy-ssh	SSH proxy, host key verification, automatic password redaction in recordings.
vauban-proxy-rdp	RDP proxy, H.264 video encoding for both live streaming and recording.

1.4 Core Security Principles

Principle	Implementation
Privilege Separation	Each service runs under a dedicated system account. Security-critical operations (hashing, authorization, encryption) are isolated in dedicated processes, following the OpenSSH model.
Capsicum Sandboxing (FreeBSD)	After entering capability mode (cap_enter), each service can only access pre-opened file descriptors. No new network connections, file system access, or socket creation is possible.
Fail-Closed	If an authorization service becomes unavailable, all access is denied. There is no permissive degraded mode in production.
In-Memory Secret Protection	Credentials are encrypted at rest, transmitted in memory via IPC, and cryptographically zeroed on destruction. They are never written in plaintext to logs or disk.
Network-Free Internal Communication	All inter-service communication uses pre-established Unix IPC channels. No internal network ports are exposed between services.

2. Actors and Personas

System Role	Persona	Profile	Scope of Action
superuser	Super-Administrator	Security officer or infrastructure team	Full access to all features and resources. The only role that can perform certain critical operations (e.g., creating the first account via CLI).
staff	Operator / IT Administrator	Operations team	Manages users, assets, groups, access rules, and sessions. Has access to the administration dashboard. Approves JIT requests.
user	End User	Employee, contractor, or developer	Views the assets they are authorized to access, opens SSH or RDP sessions, manages their own profile. Cannot see assets outside their authorized scope.
System Administrator	(Operational Persona)	Infrastructure / SRE team	Deploys and configures Vauban, manages TLS certificates, runs CLI commands, and monitors processes.

System Role	Persona	Profile	Scope of Action
			Interacts through configuration files and the terminal.

3. Authentication and Account Management

3.1 Authentication Flow

Authentication proceeds in two successive phases: verification of the username/password pair, then verification of the MFA code if the user has enabled multi-factor authentication. These operations are handled by isolated dedicated services so that the web server never handles sensitive data in plaintext.

Step	Actor	Action	Outcome
1	User	Enters username and password on /login	Form submitted
2	vauban-web	Looks up the user in the database by username	User found or generic error returned
3	vauban-auth	Securely verifies the password against the stored hash	Valid or invalid
4a	vauban-web	Invalid password	Authentication error (generic message, no account existence hint)
4b	vauban-web	User has no MFA enabled	JWT session token issued as a secure cookie, redirect to dashboard
4c	vauban-web	User has MFA enabled	Redirect to /login/mfa
5	User	Enters the 6-digit TOTP code	Form submitted
6	vauban-vault	Decrypts the TOTP secret and verifies the code	Valid or invalid
7a	vauban-web	Invalid code	MFA error
7b	vauban-web	Valid code	JWT session token issued, session created, redirect to dashboard

3.2 User Stories — Authentication and Accounts

ID	User Story	Acceptance Criteria
AUTH-01	As a user, I want to log in with my username and password.	Password securely verified by the dedicated authentication service. JWT token issued as an HttpOnly/SameSite secure cookie. Redirect to dashboard.
AUTH-02	As a user with MFA enabled, I want to be prompted for my TOTP code after validating my password.	Redirect to /login/mfa after successful primary authentication. Code verified via the secrets management service. Session created only if MFA code is valid.
AUTH-03	As a user, I want to log out at any time.	POST /auth/logout. Server-side session invalidated. Cookie cleared.
AUTH-04	As a user, I want to view and update my profile.	GET/POST /accounts/profile. Password changes processed by the dedicated authentication service.
AUTH-05	As a staff member, I want to create a user account.	GET/POST /accounts/users. Role assigned at creation. Password hashed by the dedicated authentication service.
AUTH-06	As a staff member, I want to list and view user accounts.	GET /accounts/users — paginated list. GET /accounts/users/:uuid — account detail.
AUTH-07	As a staff member, I want to update a user account (role, status, email).	POST /accounts/users/:uuid. Enforced by RBAC check (resource: users, action: write).
AUTH-08	As a superuser, I want to create the first account via CLI without a web interface.	vauban-supervisor create-superuser. Initial bootstrap without dependency on the web interface.
AUTH-09	As a superuser, I want to reset a user's password via CLI.	vauban-supervisor reset-password <username>. New secure hash stored.
AUTH-10	As a superuser, I want to disable MFA for a locked-out user via CLI.	vauban-supervisor reset-2fa <username>. The only MFA disablement mechanism — no web interface allows this operation.

3.3 Business Rules — Authentication

ID	Rule
RG-AUTH-01	All passwords are securely hashed before storage, using parameters aligned with current security standards (configurable memory, iterations, and parallelism). The raw hash is never exposed outside the authentication service.
RG-AUTH-02	Password hashing and verification are performed exclusively by vauban-auth, a dedicated isolated service. The web server never handles passwords in plaintext. If this service is unavailable, all login attempts are refused.
RG-AUTH-03	Any secret handled in memory (plaintext password, TOTP code, private key) is wrapped in a specialized type that guarantees automatic cryptographic zeroing on destruction and systematic masking in logs and debug output.

ID	Rule
RG-AUTH-04	MFA uses the TOTP protocol (RFC 6238). The TOTP secret is encrypted at rest in the database by the secrets management service. If found in plaintext (legacy migration), it is automatically re-encrypted on the next successful login.
RG-AUTH-05	Disabling MFA is only possible via the CLI command executed by a superuser. No web interface allows this operation, to prevent accidental or malicious deactivation.
RG-AUTH-06	After successful authentication, a JWT token is issued and stored as a secure cookie (HttpOnly, SameSite). Its validity duration is configurable in the main configuration file.

3.4 RBAC Permission Matrix

The RBAC policy is evaluated by vauban-access using a rules engine loaded from a versioned policy file. It controls access to features (feature-level), independently of asset-level access rules.

Resource	Action	superuser	staff	user
users	read / write	✓ / ✓	✓ / ✓	— / —
assets	read / write	✓ / ✓	✓ / ✓	✓ / —
sessions	read / write / create	✓ / ✓ / ✓	✓ / ✓ / ✓	✓ / — / ✓
groups	read / write	✓ / ✓	✓ / ✓	— / —
access_rules	read / write	✓ / ✓	✓ / ✓	— / —
admin	view	✓	✓	—
profile	read / write	✓ / ✓	✓ / ✓	✓ / ✓

4. Asset Management

4.1 Definition and Types

An asset is a target computing resource that users connect to through the bastion. Two types are supported: Unix/Linux servers or devices accessible via SSH, and Windows workstations or servers accessible via RDP. No direct connection to an asset is possible without routing through Vauban.

4.2 Data Model — Asset

Attribute	Type	Description
uuid	uuid (UK)	Public identifier exposed in the API and URLs.

Attribute	Type	Description
name	varchar(255)	Human-readable asset name (e.g., 'Production Web Server').
description	text	Free-form description, optional.
asset_type	varchar	Protocol type: 'ssh' or 'rdp'. Immutable after creation.
group_id	integer	Reference to the asset group. Referential integrity is enforced at the application layer to allow database isolation between services.
connection_config	jsonb	Connection parameters: host, port, encrypted credentials, SSH host key. Structure varies by asset type.
is_deleted	bool	Logical deletion flag. The asset remains in the database for the audit history of past sessions.
created_by_id / updated_by_id	integer (FK)	Authors of creation and last modification.
created_at / updated_at	timestampz	Creation and modification timestamps.

SSH asset — connection_config fields

Field	Description
host / port	IP address or hostname of the target server. SSH port (default: 22).
username	Username for the SSH connection.
auth_method	Authentication method: 'password' or 'key'.
password	SSH password. Encrypted by the secrets management service before storage.
private_key / passphrase	SSH private key (PEM format) and optional passphrase. Encrypted by the secrets management service.
ssh_host_key	Expected server public key (OpenSSH format). Used to detect MITM attacks on every connection.

RDP asset — connection_config fields

Field	Description
host / port	IP address or hostname of the RDP server. RDP port (default: 3389).
username	Windows username.
password	Windows password. Encrypted by the secrets management service.
domain	Active Directory domain (optional).

4.3 Asset Groups

Assets are organized into groups. An asset belongs to zero or one group. Groups are the management unit for access rules: a rule grants a user group access to an asset group, not to individual assets.

Attribute	Description
name / slug	Human-readable name and URL-friendly unique identifier of the group (e.g., 'Production', 'production').
description / color / icon	Display information shown in the web interface.
parent_id	Parent group for hierarchical organization (planned for a future release).

4.4 User Stories — Assets

ID	User Story	Acceptance Criteria
ASS-01	As a user, I want to see only the assets I am authorized to access, with the appropriate connection buttons.	The list is filtered to groups accessible through active access rules. SSH/RDP buttons appear only for authorized protocols. Staff and superusers see all assets.
ASS-02	As a staff member, I want to create an SSH asset with its connection parameters.	Required fields: name, asset_type='ssh', host, port, username, auth_method. Credentials are encrypted by the secrets management service before storage. HTTP 201 returned.
ASS-03	As a staff member, I want to create an RDP asset with its connection parameters.	Required fields: name, asset_type='rdp', host, port, username, password. Password encrypted by the secrets management service.
ASS-04	As a staff member, I want to update an existing asset (name, configuration, group).	POST /assets/:uuid/edit. Modified connection fields are re-encrypted. Modification timestamps are automatically updated.
ASS-05	As a staff member, I want to create and manage asset groups.	Full CRUD on asset groups through the access control service. Name and slug are required; slug must be unique and URL-friendly.
ASS-06	As a staff member, I want to retrieve and store the SSH host key of an asset to prevent MITM attacks.	GET /assets/:uuid/verify-host-key. POST /assets/:uuid/fetch-host-key?confirm=true. The SSH proxy establishes a minimal connection and extracts the server's public key. The fingerprint is displayed to the administrator for manual verification before acceptance.
ASS-07	As a staff member, I want to view the SSH host key status of an asset.	GET /assets/:uuid. Returns 'verified' (matching key), 'mismatch' (changed key), or 'no_key' (no key stored).
ASS-08	As a staff member, I want to explicitly accept a changed host key after verification.	POST /assets/:uuid/fetch-host-key?confirm=true. Requires staff or superuser role. The action is logged by the audit service.

4.5 Business Rules — Assets

ID	Rule
RG-ASS-01	SSH/RDP credentials (password, private key, passphrase) are encrypted by the secrets management service before any storage. They are never stored in plaintext in production. Encrypted values use a versioned format that supports key rotation.
RG-ASS-02	Credential decryption is performed exclusively by the secrets management service, on demand from the proxy, immediately before SSH/RDP authentication. Decrypted credentials do not persist in memory beyond their immediate use.
RG-ASS-03	On every SSH connection, the SSH proxy compares the host key presented by the server against the stored value. A mismatch results in an immediate connection refusal, an audit event, and an error message returned to the user.
RG-ASS-04	Asset deletion is a logical deletion (soft delete): the <code>is_deleted</code> flag is set to true and the deletion timestamp is recorded. The asset is no longer visible in listings but its data is retained for the audit history of past sessions.
RG-ASS-05	Referential integrity between the assets table and the asset groups table is enforced at the application layer, to allow running separate database instances for different services.

5. Access Control and Groups

5.1 Two-Layer Authorization Model

Vauban implements a two-layer authorization model with independent and complementary layers, both evaluated by `vauban-access`, a dedicated isolated authorization service:

Layer	Scope	Question Answered	Example
1 — RBAC	Feature-level	Can this role use this feature?	Can the 'staff' role manage access rules?
2 — Instance-level	Asset-level	Can this user connect to this asset?	Can John open an SSH session on the 'Production' group?

Both layers are systematically evaluated for every connection attempt. An RBAC authorization does not imply instance-level access, and vice versa. `vauban-access` is the sole authorization decision-maker in production.

5.2 User Groups

Users are organized into groups. Access rules apply to a user group — never to an individual user. A user may belong to multiple groups.

Attribute	Description
name	Group name (e.g., 'DevOps Team', 'Administrators').
description	Free-form description.
source	'local' (manually managed group) or 'ldap'/'oidc' (synchronized from an external directory).
external_id	Group identifier in the external directory (LDAP DN, OIDC group claim).
last_synced	Timestamp of the last synchronization from the external directory.

5.3 Access Rules (access_rules)

An access rule links a user group to an asset group with a set of constraints. It is the fundamental configuration unit for instance-level access control.

Attribute	Type	Description
name	varchar(100)	Human-readable rule name (e.g., 'DevOps -> Production SSH').
user_group_id	integer (FK)	The user group this rule applies to.
asset_group_id	integer (FK)	The asset group this rule grants access to.
allowed_protocols	text[]	Allowed protocols: ['ssh'], ['rdp'], or ['ssh', 'rdp'].
valid_from / valid_until	timestampz	Validity time window. NULL means no constraint.
require_mfa	bool	MFA is mandatory for all connections governed by this rule.
require_approval	bool	Prior approval (JIT workflow) is required before any connection.
max_session_duration	integer	Maximum session duration in minutes. NULL means unlimited.
is_active	bool	Whether the rule is active. An inactive rule is never evaluated.
priority	integer	Priority for conflict resolution between rules. Higher value means higher priority.

5.4 User Stories — Access Control and Groups

ID	User Story	Acceptance Criteria
ACC-01	As a staff member, I want to create and manage user groups.	Full CRUD on groups through the access control service. Name is required; source defaults to 'local'.

ID	User Story	Acceptance Criteria
ACC-02	As a staff member, I want to add or remove users from a group.	Add and remove members. Existence of the user and group is verified.
ACC-03	As a staff member, I want to create an access rule linking a user group to an asset group.	POST /assets/access. Required fields: name, user_group_id, asset_group_id, allowed_protocols (at least one protocol).
ACC-04	As a staff member, I want to define a validity time window for a rule.	Optional valid_from and/or valid_until (ISO 8601 with timezone). The rule is only evaluated within those dates.
ACC-05	As a staff member, I want to require MFA for connections under a sensitive rule.	require_mfa = true. MFA is mandatory at connection time if at least one matching rule requires it.
ACC-06	As a staff member, I want to require prior approval for high-risk connections.	require_approval = true. Connections under this rule trigger the JIT workflow and require explicit approval from a staff member or superuser.
ACC-07	As a staff member, I want to cap the maximum duration of a session.	max_session_duration in minutes. The session is automatically terminated at expiration, without user action.
ACC-08	As a staff member, I want to disable a rule without deleting it.	POST /assets/access/:uuid/edit. The rule remains in the database but is never evaluated.

5.5 JIT Workflow — Access Request and Approval

The Just-In-Time workflow is triggered when the matching access rule has `require_approval=true`. It enforces human approval before any connection can be established.

Status	Trigger	Possible Transitions
pending	User submits an access request with justification	→ approved (staff approves) / → expired (24 hours without approval)
approved	Staff approves the request (with optional duration override)	→ connecting (user clicks 'Connect')
connecting	Connection being established	→ active (communication channel opened) / → disconnected (failure)
active	SSH or RDP channel established; connected_at and expires_at calculated	→ disconnected (voluntary disconnect) / → terminated (max duration reached or admin action)
disconnected	User closes the session	— (terminal state)
terminated	Automatic expiration (periodic task every 30s) or admin termination	— (terminal state)
expired	Pending request not approved within 24 hours	— (terminal state)

Step	Actor	Action	Outcome
1	User	GET /assets/:uuid#request-access (asset_uuid, justification)	Session created: status='pending', max duration copied from the access rule
2	Staff	Reviews the list of pending requests	Filtered by status='pending'
3a	Staff	Approves with optional duration (value + unit)	status='approved', approver identity and timestamp recorded
3b	Auto task (30s)	Request pending for over 24 hours	status='expired'
4	User	Clicks 'Connect SSH/RDP'	status='connecting'
5	vauban-web + proxies	SSH or RDP channel successfully opened	status='active', connected_at=NOW(), expires_at calculated
6a	User	Closes the session	status='disconnected'
6b	Auto task (30s)	expires_at <= NOW()	status='terminated', disconnected_at=NOW()
6c	Staff	POST /sessions/:id/terminate	status='terminated'

5.6 Business Rules — Access Control

ID	Rule
RG-ACC-01	An access rule is evaluated only when all five conditions are simultaneously met: (1) the user is a member of the rule's user group, (2) the target asset belongs to the asset group, (3) the rule is active, (4) the current date falls within the validity window, (5) the requested protocol is in the list of allowed protocols.
RG-ACC-02	If no rule matches, access is denied (fail-closed). There is no default access. vauban-access is the sole decision-maker in production — the web server never makes instance-level access decisions autonomously.
RG-ACC-03	When multiple rules match for the same (user, asset group, protocol) triplet, constraints are aggregated using the most restrictive logic: require_mfa and require_approval are true if at least one matching rule requires them; max_session_duration is the minimum across all matching rules.
RG-ACC-04	The maximum session duration flows as follows: source value in the access rule → copied to the session at creation → optionally overridden by a staff member during JIT approval (between 1 minute and 24 hours) → converted to an expiration timestamp when the connection is activated.
RG-ACC-05	The automatic session expiration task runs every 30 seconds. The task that expires JIT requests pending for more than 24 hours runs on the same schedule.

6. Sessions and Recording

6.1 Data Model — Proxy Sessions

Attribute	Type	Description
uuid	uuid (UK)	Public identifier exposed in the API and URLs.
user_id	integer (FK)	User who opened the session.
asset_id	integer (FK)	Target asset for the session.
session_type	varchar	Type: 'ssh' or 'rdp'.
status	varchar	Current status: pending, approved, connecting, active, disconnected, terminated, expired.
justification	text	Justification provided by the user when submitting a JIT request.
approved_by_id / approved_at	integer / timestamptz	Identity and timestamp of JIT approval.
max_session_duration	integer	Maximum duration in seconds. Copied from the access rule; optionally overridden at approval.
connected_at	timestamptz	Timestamp when the communication channel was effectively established (transition to status='active').
expires_at	timestamptz	Calculated expiration: connected_at + max_session_duration. NULL if no limit.
disconnected_at	timestamptz	Session end timestamp.
is_recorded	bool	Recording available. Set to true on channel disconnect.
recording_path	text	Relative path to the recording directory (e.g., '2026/03/uuid/').
source_ip	inet	User's source IP address.

6.2 SSH Session Flow

Step	Actor	Action	Outcome
1	User	Clicks 'Connect SSH' on the asset page	GET /sessions/terminal/:asset_uuid
2	vauban-web	RBAC check + instance-level access check	Authorized with constraints (require_mfa, require_approval, max_session_duration)
3	vauban-supervisor	DNS resolution + TCP connection to the target asset	TCP socket connected
4	vauban-supervisor	Connected socket transmitted to the SSH proxy via file descriptor passing (SCM_RIGHTS)	Socket handed to the proxy

Step	Actor	Action	Outcome
5	vauban-proxy-ssh	SSH handshake, received host key compared against stored key	Match or mismatch → connection refused + audit event
6	vauban-proxy-ssh	Credentials retrieved from the secrets management service, SSH authentication	SSH session established on the target server
7	vauban-proxy-ssh	Recording started → audit service (session.cast file created by supervisor)	Recording in progress
8	User	Connects to the web terminal via WebSocket	connected_at and expires_at calculated, status='active'
9	Ongoing	Bidirectional terminal data. Recording events transmitted to the audit service in real time	Interactive session + continuous recording
10	User	Closes the terminal	Channel closed, recording finalized, status='disconnected', is_recorded=true

6.3 RDP Session Flow

Step	Actor	Action	Outcome
1	User	Clicks 'Connect RDP' on the asset page	GET /sessions/rdp/:asset_uuid
2	vauban-web	RBAC check + instance-level access check	Authorized with constraints
3	vauban-supervisor	DNS resolution + TCP connection to the asset's RDP port	TCP socket connected
4	vauban-supervisor	Connected socket transmitted to the RDP proxy via file descriptor passing	Socket handed to the proxy
5	vauban-proxy-rdp	RDP handshake (TLS negotiation + network authentication), credentials retrieved	RDP session established on the target server
6	vauban-proxy-rdp	Recording started → audit service (first fMP4 segment file created by supervisor)	Recording in progress
7	User	Connects to the remote desktop via WebSocket	connected_at and expires_at calculated, status='active'
8	Ongoing	Encoded H.264 video frames → web server (live stream) AND audit service (recording)	Simultaneous dual dispatch

Step	Actor	Action	Outcome
9	User	Resizes the browser window	Resize signal → proxy → RDP server → new fMP4 segment if resolution changes
10	User	Closes the remote desktop	Channel closed, recording finalized with metadata, status='disconnected', is_recorded=true

6.4 Session Recording

SSH Format — AsciiCast v2

SSH sessions are recorded in the AsciiCast v2 format (open standard, newline-delimited JSON text). The recording file contains a JSON header line followed by one line per timestamped terminal event. A metadata file accompanies each recording and contains the integrity hash computed over the entire file.

Element	Format	Example
Header	JSON object (first line)	<code>{"version":2,"width":120,"height":40,"timestamp":1710700000,"title":"SSH: user@asset"}</code>
Output event	JSON array	<code>[0.500000, "o", "\$ ls -la\r\n"]</code>
Redacted input event	JSON array	<code>[5.100000, "i", "[REDACTED]\r\n"]</code>
Resize event	JSON array	<code>[12.000000, "r", "160x50"]</code>

Automatic SSH Password Redaction

The SSH proxy's redaction component detects and replaces passwords before transmitting data to the audit service. Two independent detection layers are used; either one alone is sufficient to trigger suppression:

Layer	Mechanism	Detection Examples
1 — Pattern Matching	Case-insensitive search in server output for a hardcoded, non-configurable list of patterns characteristic of password prompts.	'Password:', '[sudo] password for user:', 'Enter passphrase:', 'Verification code:', 'Token:'

Layer	Mechanism	Detection Examples
2 — Echo Suppression Detection	Automatic detection when 3 or more printable characters are typed without being echoed back by the server (threshold designed to avoid false positives with text editors).	Any custom prompt or non-English prompt that disables terminal echo.

While suppression is active, typed characters are silently accumulated. When Enter is pressed, a single '[REDACTED]' event is recorded. The suppression state resets after each submission.

RDP Format — Segmented fMP4

Aspect	Description
Container	ISO BMFF / fMP4 (Fragmented MP4, ISO 14496-12). Each encoding unit (GOP) is immediately flushed to disk — the file remains readable and recoverable even after an abrupt process termination.
Video codec	H.264 (AVC, ITU-T H.264 / ISO 14496-10). Encoding is performed by the RDP proxy; raw encoded data is forwarded to the audit service without re-encoding.
Segmentation	A new segment file is created on every desktop resolution change. A grace period is applied after each resolution change to allow the display buffer to fully reconstruct before the first encoded frame.
Integrity hash	Cryptographic hash computed per segment (RDP) or per session (SSH), stored in a metadata file. Can be recomputed by the administrator to detect any post-capture modification.
Segmented playback	DASH standard (Dynamic Adaptive Streaming over HTTP, ISO 23009-1). A manifest is dynamically generated from recording metadata. Resolution transitions are transparent to the viewer.
Legacy playback	Pre-segmentation single-file recordings are served with the HTTP Range protocol and played natively by the browser. Full backward compatibility is maintained.

6.5 User Stories — Sessions and Recording

ID	User Story	Acceptance Criteria
SES-01	As a user, I want to open an interactive SSH session to an authorized asset.	RBAC + instance-level checks. TCP connection established by supervisor and passed to proxy. Host key verified. Interactive terminal via WebSocket.
SES-02	As a user, I want to open an RDP session to an authorized asset.	Same flow as SSH. Remote desktop streamed as H.264 video via WebSocket. Dynamic resizing supported.

ID	User Story	Acceptance Criteria
SES-03	As a staff member, I want to list all sessions with their status.	GET /sessions — all sessions, filterable by status and type. For the 'user' role: filtered to the user's own sessions.
SES-04	As a staff member, I want to terminate an active session.	POST /sessions/:id/terminate. status → 'terminated', disconnected_at = NOW(). Logged by the audit service.
REC-01	As a system, active SSH sessions are automatically recorded in Ascicast v2 with password redaction.	Start, data, and end events transmitted to the audit service. session.cast file and metadata file with integrity hash produced.
REC-02	As a system, active RDP sessions are automatically recorded as segmented fMP4.	Video frames dispatched simultaneously for live viewing and recording. Segmented by resolution. Metadata file with per-segment integrity hash.
REC-03	As a staff member, I want to play back an SSH session recording as an interactive terminal.	GET /sessions/recordings/:uuid/play. Played via an integrated interactive terminal player. Administrator access only.
REC-04	As a staff member, I want to play back an RDP session recording as a video.	GET /sessions/recordings/:uuid/play. (DASH manifest dynamically generated from metadata). Transparent resolution transitions. Administrator access only.

6.6 Business Rules — Sessions and Recording

ID	Rule
RG-SES-01	Any SSH or RDP connection requires two prior checks, both handled by vauban-access: the RBAC check (can this role create sessions?) and the instance-level access check (is this user authorized on this asset group with this protocol?).
RG-SES-02	The TCP connection to the target asset is established exclusively by vauban-supervisor, which is not sandboxed. The connected socket is passed to the proxy via file descriptor passing (SCM_RIGHTS). The proxies, sandboxed via Capsicum (FreeBSD), have no direct network access — they cannot resolve DNS names or open sockets.
RG-REC-01	The recording and playback services cannot directly access the file system, as they are sandboxed via Capsicum (FreeBSD). All recording files are created and opened by vauban-supervisor, which passes the file descriptors to the respective services via SCM_RIGHTS.
RG-REC-02	Passwords typed in an SSH terminal are redacted in the SSH proxy before any data is transmitted to the audit service. Sensitive data never reaches the audit service and is never written to disk.
RG-REC-03	Recording playback is restricted to administrators (staff and superuser). Access is conditioned on the session existing in the database with is_recorded=true and a non-null recording_path.
RG-REC-04	The integrity hash is computed incrementally during recording and stored in a metadata file at the end of each session. It enables detection of any post-capture modification of recording files.

7. Infrastructure and Administration

7.1 Process Orchestration

vauban-supervisor is Vauban's root process. It starts all services in dependency order, creates all IPC channels before any process fork, monitors each service's health via a heartbeat protocol, and automatically restarts services upon failure.

Startup Phase	Services Started	Prerequisites
Phase 1 — Supervisor	vauban-supervisor	Reads configuration, creates IPC channels, manages TLS certificates.
Phase 2 — Base Services	vauban-audit, vauban-vault, vauban-access	No inter-service dependencies. Started in parallel.
Phase 3 — Authentication	vauban-auth	Depends on the access control service (role verification) and the secrets management service.
Phase 4 — Proxies	vauban-proxy-ssh, vauban-proxy-rdp	Depend on the access control, secrets management, and audit services.
Phase 5 — Web	vauban-web	Depends on all other services.

Watchdog Parameters

Parameter	Default	Description
heartbeat_interval_secs	5	Frequency of liveness pings sent to each child service.
heartbeat_timeout_secs	2	Maximum wait time for a response.
max_missed_heartbeats	3	Number of consecutive missed responses before declaring failure and triggering a restart.
max_respawns_per_hour	10	Maximum restarts per hour per service. Beyond this threshold, manual intervention is required.
drain_timeout_secs	30	Time allowed for a service to finish in-progress requests before a forced shutdown.

Linked Restart Groups

Services sharing an anonymous IPC channel form a restart group. If one crashes, the channel is broken and sandboxed services cannot open a new one. All members of the group must be restarted together to restore communication.

Group	Member Services	Shared Channel
SSH Group	vauban-web + vauban-proxy-ssh	Bidirectional terminal data stream.
RDP Group	vauban-web + vauban-proxy-rdp	RDP video frame stream.

7.2 TLS Certificates and ACME

Vauban manages its TLS certificates in a fully automated manner, without any external dependencies. ACME validation uses exclusively the TLS-ALPN-01 protocol (RFC 8737) on port 443, which is already used by the web server — port 80 is never required.

Step	Actor	Action	Outcome
1	Scheduler (vauban-web)	Detects that the renewal date has been reached (notAfter - 24h) or that the current certificate is self-signed	Renewal triggered
2	vauban-web	Renewal request sent to the supervisor via IPC	Request forwarded
3	vauban-supervisor	ACME order created with the certificate authority. TLS-ALPN-01 challenges obtained	Token and authorization per domain
4	vauban-supervisor	Ephemeral self-signed challenge certificate generated with the acmelfield identifier extension (OID 1.3.6.1.5.5.7.1.31)	Challenge certificate ready
5	vauban-supervisor	Challenge certificate installed in vauban-web's TLS resolver via IPC	Challenge active
6	Certificate Authority	TLS connection to domain:443 with ALPN 'acme-tls/1' — acmelfield identifier extension validated	Domain validated
7	vauban-supervisor	Challenge certificates removed — order finalized — certificate chain downloaded	cert.pem + key.pem obtained
8	vauban-supervisor	Atomic write to disk (temp file + fsync + atomic rename, restrictive permissions)	Certificates safely persisted
9	vauban-supervisor	New certificate activated in memory in vauban-web's TLS resolver via IPC	Zero downtime — new TLS connections immediately use the new certificate

Key points: vauban-web never reads certificate files from disk in production — they are transmitted via IPC at startup (initial provisioning) and on each renewal (activation). The certificate directory is accessible in read/write mode only by the supervisor. The renewal

scheduler uses a precise sleep (until notAfter - renewal_lead_time) for zero resource consumption between renewals. Any certificate authority compliant with RFC 8555 is supported.

7.3 Configuration

Vauban uses TOML configuration files with a strategy adapted to the runtime environment. In development mode, files are layered. In production (release build), a single self-contained file is loaded.

Profile	Files Loaded	Usage
Development	default.toml + development.toml + local.toml (optional, unversioned)	Local development. Privilege separation can be disabled, verbose logs, self-signed certificates.
Testing	default.toml + testing.toml	Automated tests. Dedicated test database.
Production	/usr/local/etc/vauban/vauban.conf (single file)	FreeBSD deployment. Always in production mode — the environment variable is ignored (compiled out of the binary).

Main Configuration Sections

Section	Key Parameters
[supervisor]	privsep (privilege separation toggle), heartbeat_interval_secs, max_respawns_per_hour, drain_timeout_secs
[services.*]	name, binary, uid, gid — one block per service (audit, vault, access, auth, proxy_ssh, proxy_rdp, web)
[database]	url (PostgreSQL connection string), max_connections — fixed connection pool, pre-established before entering sandbox
[cache]	url (Valkey/Redis), enabled — if disabled or unavailable: a no-op mock cache is automatically activated
[server.tls.acme]	enabled, email, domains, renew_before_hours, directory_url (ACME directory URL), staging, eab_kid, eab_hmac_key (EAB for ZeroSSL)
[auth]	Secure password hashing parameters (memory, iterations, parallelism) — injected into vauban-auth via environment variables
[access]	Paths to RBAC model and policy files. Optional database URL (separate instance supported)
[recording]	enabled (global toggle), storage_path (storage directory), rdp (bool), ssh (bool) — per-protocol switches

7.4 CLI Administration Tools

Command	Usage	Description
create-superuser	vauban-supervisor create-superuser	Creates the first superuser account. Password entered interactively and securely hashed.
reset-password	vauban-supervisor reset-password <username>	Resets an existing user's password. New secure hash stored.
reset-2fa	vauban-supervisor reset-2fa <username>	Disables MFA for a user. The only available mechanism — no web interface allows this.
seed-data	vauban-supervisor seed-data	Populates the database with predefined test data (development and demo environments).
migrate-secrets	vauban-supervisor migrate-secrets [--dry-run]	Bulk-encrypts credentials stored in plaintext. Idempotent: already-encrypted values are skipped. --dry-run previews the operation without modifying the database.

7.5 User Stories — Infrastructure and Administration

ID	User Story	Acceptance Criteria
INF-01	As a system administrator, I want to start Vauban by launching a single binary.	The supervisor orchestrates the startup of all services in dependency order. IPC channels are created before any process fork.
INF-02	As a system administrator, I want to apply a new configuration without interrupting active sessions.	SIGHUP → configuration validation → drain of in-progress requests (timeout 30s) → ordered restart. Invalid configuration → signal ignored, system unchanged.
INF-03	As a system administrator, I want a crashed service to restart automatically.	Failure detected by watchdog. Restart with new IPC channels if needed. Linked service group restarted together. Maximum 10 restarts per hour.
INF-04	As a system administrator, I want the TLS certificate to be renewed automatically before expiration.	Precise scheduler triggered at notAfter - 24h. Zero downtime: in-memory activation without service restart. No manual intervention required.
INF-05	As a system administrator, I want the TLS private key to never be accessible to the web service on the file system.	The supervisor transmits PEM data via IPC. The certificate directory is only accessible by the supervisor.

7.6 Business Rules — Infrastructure

ID	Rule
RG-INF-01	Each service runs under a dedicated unprivileged system account. The supervisor holds elevated privileges temporarily to create IPC channels and start child processes, then drops them as soon as possible. No application service runs with elevated privileges in steady state.
RG-INF-02	After entering Capsicum capability mode (FreeBSD), no sandboxed service can open a new file descriptor, create a socket, or access the file system. Any operation requiring network or file system access is delegated to the supervisor via IPC.
RG-INF-03	If a service loses its database connection after entering sandbox mode (opening a new connection is not possible), it exits with a conventional exit code signaling the supervisor to restart it immediately to restore connections.
RG-INF-04	A graceful restart (SIGHUP) validates the new configuration before applying it. An invalid configuration is rejected — the system continues operating with the previous configuration without service interruption.
RG-INF-05	The secret migration operation (migrate-secrets) is idempotent: already-encrypted credentials are identified by their format and skipped without modification. The operation can be safely re-run in case of a partial interruption.

8. Cross-Cutting Non-Functional Requirements

ID	Category	Requirement
NFR-01	Security	No application service runs with elevated privileges in steady state. Each service has a dedicated system account. The supervisor drops its elevated privileges as soon as possible after startup.
NFR-02	Security	Sensitive data (passwords, private keys, TOTP secrets) is never stored in plaintext. Systematic encryption by the secrets management service, automatic cryptographic zeroing in memory, masking in logs.
NFR-03	Security	SSH passwords are redacted in the SSH proxy before any data is transmitted to the audit service. They never reach the audit service and are never written to disk.
NFR-04	Security	SSH host key verification is mandatory before any session is established. A mismatch results in an immediate connection refusal and an audit event.
NFR-05	Security	The RBAC authorization policy is loaded before entering sandbox mode and cannot be modified at runtime. Any change requires a restart of the access control service.
NFR-06	Isolation	After entering Capsicum capability mode (FreeBSD), no sandboxed service can access the file system, create sockets, or establish network connections. Any such operation is delegated to the supervisor via IPC.
NFR-07	Isolation	All inter-service communication uses pre-established Unix IPC channels. No internal network ports are exposed between services.

ID	Category	Requirement
NFR-08	Availability	TLS certificate renewal is zero-downtime: the new certificate is activated in memory without a service restart or interruption of ongoing connections.
NFR-09	Availability	The watchdog detects unresponsive services and restarts them automatically, up to 10 times per hour per service.
NFR-10	Availability	The caching service can be disabled or unavailable without impacting critical functionality: a no-op fallback is automatically activated.
NFR-11	Resilience	The fMP4 format is crash-resilient: each encoding unit is immediately flushed to disk. In case of abrupt termination, all previously written units remain playable.
NFR-12	Resilience	If a service loses its database connection after entering sandbox mode, it exits with a conventional code to trigger an automatic restart by the supervisor.
NFR-13	Performance	Recording playback is streamed in fixed-size chunks: memory consumption is constant regardless of file size.
NFR-14	Performance	RDP dual dispatch (live streaming and recording) is asynchronous: live session latency is not affected by recording write operations.
NFR-15	Auditability	Every RBAC and instance-level authorization decision is written to a structured log. Every significant session event generates an audit event transmitted to the audit service.
NFR-16	Integrity	Every recording is associated with a cryptographic integrity hash stored in a metadata file, enabling detection of any post-capture modification.

9. Consolidated API Reference

9.1 Authentication and Accounts

Method	Endpoint	Required Role	Description
POST	/api/v1/auth/login	— (public)	Authentication. Returns a JWT cookie on success.
POST	/api/v1/auth/logout	Authenticated	Logout. Invalidates the server-side session.
GET	/api/v1/accounts	staff, superuser	Paginated list of user accounts.
POST	/api/v1/accounts	staff, superuser	Create a user account.
GET	/api/v1/accounts/:uuid	staff, superuser, self	Account detail.
PUT	/api/v1/accounts/:uuid	staff, superuser, self	Update account (role, email, password).
DELETE	/api/v1/accounts/:uuid	—	Not implemented (HTTP 501).

9.2 Assets and Asset Groups

Method	Endpoint	Required Role	Description
GET	/api/v1/assets	staff (user: filtered)	Paginated list. Filtered by access rights for the 'user' role.
POST	/api/v1/assets	staff, superuser	Create an asset. Credentials encrypted via the secrets management service.
GET	/api/v1/assets/:uuid	staff, authorized user	Asset detail.
PUT	/api/v1/assets/:uuid	staff, superuser	Update an asset. Credentials re-encrypted if modified.
DELETE	/api/v1/assets/:uuid	—	Not implemented (HTTP 501).
GET	/api/v1/assets/:uuid/ssh-host-key	staff, superuser	SSH host key status: 'verified', 'mismatch', or 'no_key'.
POST	/api/v1/assets/:uuid/ssh-host-key	staff, superuser	Retrieve and store the host key from the target server.
POST	/api/v1/assets/:uuid/ssh-host-key?confirm=true	staff, superuser	Accept a changed host key after manual verification.
GET	/api/v1/assets/groups	staff, superuser	Paginated list of asset groups.
GET	/api/v1/assets/groups/:uuid/assets	staff, superuser	Assets belonging to a group.

9.3 User Groups and Access Rules

Method	Endpoint	Required Role	Description
GET	/api/v1/groups/:uuid/members	staff, superuser	List of group members (read-only).
GET	/api/v1/access-rules	staff, superuser	Paginated list of access rules.
POST	/api/v1/access-rules	staff, superuser	Create an access rule.
GET	/api/v1/access-rules/:uuid	staff, superuser	Access rule detail.
PUT	/api/v1/access-rules/:uuid	staff, superuser	Update an access rule.
DELETE	/api/v1/access-rules/:uuid	staff, superuser	Delete an access rule.

9.4 Sessions and Recordings

Method	Endpoint	Required Role	Description
GET	/api/v1/sessions	staff (user: filtered)	Paginated list of sessions.

Method	Endpoint	Required Role	Description
POST	/api/v1/sessions	user, staff, superuser	Create a session or submit a JIT access request.
GET	/api/v1/sessions/:uuid	staff, owner	Session detail.
POST	/api/v1/sessions/:id/terminate	staff, superuser	Force-terminate an active session.
POST	/sessions/:uuid/approve	staff, superuser	Approve a JIT request with optional duration override.
DELETE	/api/v1/sessions/:uuid	—	Not implemented (HTTP 501).
GET	/recordings/:uuid/session.cast	staff, superuser	Stream the AsciiCast v2 recording (SSH). Administrators only.
GET	/recordings/:uuid/manifest.mpd	staff, superuser	Dynamically generated DASH manifest (segmented RDP). Administrators only.
GET	/recordings/:uuid/:segment	staff, superuser	Individual fMP4 segment with HTTP Range support (segmented RDP).
GET	/recordings/:uuid	staff, superuser	Full recording with HTTP Range support (legacy RDP).

10. Business Glossary

Term	Definition
ACME (RFC 8555)	Automated Certificate Management Environment. Standard protocol for automatically obtaining and renewing TLS certificates from a certificate authority (Let's Encrypt, ZeroSSL, etc.).
AsciiCast v2	Open terminal session recording format (newline-delimited JSON text). Produces a file containing a session header and timestamped terminal events (server output, user input, resize).
Asset	A target computing resource (SSH server or RDP workstation/server) accessible through the Vauban bastion. No direct connection to an asset is possible without routing through Vauban.
Asset Group	A logical grouping of assets. The management unit for access rules: a rule grants a user group access to an asset group, not to individual assets.
Capsicum (FreeBSD)	Capability-based sandboxing mechanism of the FreeBSD operating system. After calling <code>cap_enter()</code> , a process can only access pre-opened file descriptors. No new file, socket, or network access is possible.
connection_config	Structured storage field containing all connection parameters for an asset: host, port, encrypted credentials, SSH host key. Its structure varies by asset type (SSH or RDP).

Term	Definition
DASH (Dynamic Adaptive Streaming over HTTP, ISO 23009-1)	Adaptive video streaming standard. Used for playing back segmented RDP recordings via a dynamically generated MPD manifest.
Fail-Closed	Vauban's default security behavior: if an authorization service becomes unavailable, all access is denied. There is no permissive degraded mode in production.
fMP4 (Fragmented MP4, ISO 14496-12)	A variant of the MP4 format in which each video encoding unit (GOP) is self-contained and immediately flushed to disk. Crash-resilient: previously written units remain playable after an abrupt process termination.
GOP (Group of Pictures)	The basic encoding unit in H.264: a key frame (IDR, independently decodable) followed by its dependent frames (P-frames). Forms a fragment in fMP4 recordings.
H.264 (AVC, ITU-T H.264 / ISO 14496-10)	Video compression standard used for encoding RDP sessions, for both live streaming to the browser and persistent recording.
Integrity Hash	A value computed by a cryptographic hash algorithm over the content of a recording, stored in the metadata file. Enables detection of any post-capture modification of recording files.
IPC (Inter-Process Communication)	Inter-service communication channels implemented as Unix pipe pairs pre-created by the supervisor. All internal communication uses these channels — no internal network ports are exposed between services.
JIT (Just-In-Time)	A temporary access mechanism requiring mandatory prior human approval. Triggered by <code>require_approval=true</code> on an access rule. Requests not approved within 24 hours expire automatically.
JWT (JSON Web Token, RFC 7519)	A signed token carrying the user's identity and role. Issued after successful authentication and stored as a secure cookie (HttpOnly, SameSite).
<code>max_session_duration</code>	The maximum duration of an active session. Defined in the access rule, copied to the session at creation, optionally overridden by a staff member during JIT approval, and enforced as a calculated expiration timestamp when the connection is activated.
MFA / TOTP (RFC 6238)	Multi-Factor Authentication via Time-based One-Time Password. A 6-digit code valid for 30 seconds, generated from a shared secret between the user and the system. Verified by the secrets management service.
PAM (Privileged Access Management)	The security discipline of controlling, recording, and auditing connections to critical systems. Vauban is an open-source PAM bastion.
RBAC (Role-Based Access Control)	Access control for features based on the user's role. Layer 1 of Vauban's authorization model, evaluated by <code>vauban-access</code> from a versioned policy file.
Redaction	Automatic replacement of sensitive data (passwords) in SSH session recordings with the '[REDACTED]' marker, performed in the SSH proxy before any data is transmitted to the audit service.
SCM_RIGHTS	A Unix mechanism for passing file descriptors between processes via a Unix socket. Used to transfer connected TCP sockets to proxies (connection brokering) and recording file descriptors to the audit service.

Term	Definition
Soft Delete	Logical deletion: a record is marked as deleted (is_deleted=true) with a timestamp, but physically remains in the database to preserve the audit history of past sessions.
TLS-ALPN-01 (RFC 8737)	An ACME validation method operating exclusively on port 443 via the X.509 acmeIdentifier extension (OID 1.3.6.1.5.5.7.1.31) and ALPN negotiation with 'acme-tls/1'. Does not require port 80.
vauban-access	The access control service, isolated and sandboxed. Evaluates RBAC (role policy loaded from a file) and instance-level access rules (database). The sole authorization decision-maker in production.
vauban-audit	The session recording service, isolated and sandboxed via Capsicum (FreeBSD). Produces fMP4 files (RDP) and AsciiCast v2 files (SSH). Never accesses the file system directly: file descriptors are provided by the supervisor.
vauban-auth	The secure password hashing and verification service, isolated and sandboxed. Stateless, no database. Processes passwords in isolation from the web server.
vauban-supervisor	Vauban's non-sandboxed orchestrator process. Starts, monitors (watchdog), and restarts child services. Brokers TCP connections to assets and recording file access.
vauban-vault	The secrets management service, isolated and sandboxed. Encrypts and decrypts SSH/RDP credentials and MFA secrets. Encryption keys are held exclusively in RAM — no network access, no database.